

Ejercicios para casa

Cómo trabajar este documento

25 ejercicios cortos, organizados por epígrafe, para practicar por tu cuenta el control de flujo. Sigue usando la misma carpeta ejercicios_bloque1 (o crea una nueva ejercicios_bloque2 con su propio entorno virtual) — un archivo .py por ejercicio.

- Todas las variables, declaradas, inicializadas y anotadas con su tipo — la variable del for es la única excepción (no se anota nunca).
- Puedes usar ya if / elif / else y for / while con total libertad: son el contenido de este bloque.
- Sigue evitando lo que aún no hemos visto: funciones propias, listas, diccionarios (llegan en el Bloque 3).
- Comprueba tú mismo el resultado ejecutando el programa y revisando que VS Code no marque ningún warning.
- Aprovecha para hacer un commit por cada ejercicio, como práctica de git.

Este documento no incluye las soluciones a propósito, igual que en el Bloque 1: la idea es que compruebes tú mismo si el resultado es el esperado.

2.1 · Condicionales

EJERCICIO 1 · Aprobado o no [Fácil]

Declara `nota: float` con un valor de tu elección. Usando if/else, imprime "Aprobado" si `nota >= 5`, o "Suspendido" en caso contrario.

EJERCICIO 2 · Clasifica el número [Fácil]

Declara `numero: int` y, con if/elif/else, imprime "Positivo", "Negativo" o "Cero" según corresponda.

EJERCICIO 3 · Clasifica en dos pasos (if anidados) [Medio]

Declara `fila: int` y `columna: int`. Sin usar `and`, determina con if ANIDADOS (un if dentro de otro) si la posición está dentro del tablero: primero comprueba si la fila está entre 0 y 7, y SOLO dentro de esa rama comprueba si la columna también lo está. Imprime un mensaje distinto para cada uno de los tres casos posibles (fila inválida, fila válida pero columna inválida, ambas válidas).

EJERCICIO 4 · ¿Año bisiesto? [Medio]

Declara `año: int`. Un año es bisiesto si es divisible entre 4 pero no entre 100, salvo que también sea divisible entre 400. Escribe esa condición combinando `and`, `or` y el operador `%`, e imprime el resultado con un if/else.

EJERCICIO 5 · Arregla la indentación [Fácil — depuración]

Este fragmento falla por un error de sangría. Corrígelo:

```
if numero > 0:
    print("Positivo")
    print("(mayor que cero)")
```

2.3 • Bucles for

EJERCICIO 6 • Tabla de multiplicar [Fácil]

Pide un número por teclado, conviértelo a int, y con un for imprime su tabla de multiplicar del 1 al 10 (una línea por cada multiplicación).

EJERCICIO 7 • Suma acumulada [Fácil]

Usando `for` y `range(1, 11)`, acumula en una variable la suma de los números del 1 al 10, y muéstrala al terminar el bucle.

EJERCICIO 8 • Cuenta las vocales [Medio]

Declara una palabra (str) y recórrela con `for`. Usando `if`, cuenta cuántas letras son vocales (a, e, i, o, u) en una variable acumuladora, sin usar `count()`.

EJERCICIO 9 • Pares con paso [Fácil]

Usando `range()` con los tres argumentos (inicio, fin, paso), imprime los números pares del 0 al 20, ambos incluidos.

2.4 • Bucles while

EJERCICIO 10 • Cuenta regresiva de lanzamiento [Fácil]

Usando `while`, cuenta en voz alta (con `print`) desde 10 hasta 1, y al terminar el bucle imprime "`¡Despegue!`".

EJERCICIO 11 • Suma hasta el centinela [Medio]

Con un `while`, pide números por teclado (convertidos a int) y acúmúlos en una suma, repitiendo mientras el número introducido no sea 0. Cuando el usuario escriba 0, muestra la suma total acumulada (sin contar el 0).

EJERCICIO 12 • Arregla el bucle infinito [Fácil — depuración]

Este fragmento nunca termina. Corrígelo para que cuente correctamente de 1 a 5:

```
contador: int = 1
while contador <= 5:
    print(contador)
```

2.5 • break y continue

EJERCICIO 13 • El primer múltiplo de 7 [Fácil]

Con un `for` y `break`, busca y muestra el primer múltiplo de 7 que sea mayor que 50 (recorre los números uno a uno desde 51 en adelante).

EJERCICIO 14 · Salta los múltiplos de 3 [Medio]

Recorre `range(1, 21)` con un `for` y, usando `continue`, imprime todos los números salvo los múltiplos de 3.

2.6 · Bucles anidados

EJERCICIO 15 · Tabla de multiplicar completa [Fácil]

Con dos bucles `for` anidados, imprime la tabla de multiplicar del 1 al 5 completa (5 filas, cada una con sus 5 resultados).

EJERCICIO 16 · Rectángulo de asteriscos [Medio]

Con dos bucles `for` anidados, imprime un rectángulo de asteriscos de 4 filas por 6 columnas.

EJERCICIO 17 · La diagonal del tablero [Reto]

Con dos bucles `for` anidados (fila y columna, ambos `range(8)`), imprime "X" cuando fila y columna coincidan (la diagonal principal del tablero) y "." en el resto de casillas, formando un tablero 8x8 completo.

EJERCICIO 18 · Esquina, borde o centro: el tablero completo [Reto]

El ejercicio más exigente del bloque, igual que el ejemplo visto en clase: con dos bucles `for` anidados (fila y columna, `range(8)`) y, DENTRO, condicionales anidados, imprime para cada casilla "Esquina", "Borde" o "Centro" según corresponda. Cada fila del tablero debe quedar en su propia línea.

2.7 · Práctica integradora

EJERCICIO 19 · Refactoriza el ejercicio del tablero [Reto]

Vuelve al tablero que construiste en el Bloque 1 (con 8 llamadas a `print()` casi idénticas) y reescríbelo usando dos bucles `for` anidados, como se hizo en clase. Comprueba que la salida es exactamente la misma que antes.

EJERCICIO 20 · Cuenta una letra [Medio]

Declara una frase (`str`) y una letra a buscar. Recorriendo la frase con `for`, cuenta cuántas veces aparece esa letra (usando `if` para comparar cada carácter).

EJERCICIO 21 · Potencias de 2 [Fácil]

Con un `for` y `range(10)`, imprime las 10 primeras potencias de 2 (desde 2^0 hasta 2^9), usando el operador `**`.

EJERCICIO 22 · Suma solo los pares [Medio]

Recorre `range(1, 21)` con un `for` y, usando `if` dentro del bucle, acumula en una variable solo la suma de los números pares. Muestra el resultado final.

EJERCICIO 23 • Simula la máquina de estados [Medio — Reto]

Retoma el esqueleto visto en clase (una variable `estado: str` que empieza en `"turno_blancas"`, un `while` con `if/elif` que alterna entre `"turno_blancas"` y `"turno_negras"`). Añade una variable `contador: int` y haz que, tras 4 turnos completos (2 de cada color), el estado pase a `"fin_partida"` y el bucle termine. En cada vuelta, imprime de quién es el turno.

EJERCICIO 24 • Piezas con Unicode [Fácil]

Declara seis variables, una por cada tipo de pieza BLANCA (rey, dama, torre, alfil, caballo, peón), usando la secuencia de escape `\u` de la tabla vista en clase — no copies y pegues el símbolo directamente. Imprímelas todas en una sola línea, separadas por espacios.

```
rey_blanco: str = "\u2654"  
dama_blanca: str = "\u2655"  
# ... completa las cuatro que faltan  
print(rey_blanco, dama_blanca, ...)
```

💡 El ejercicio 19 es de los más importantes: es literalmente el puente entre el Bloque 1 y el Bloque 2. El 23 y el 24 lo son igual de importantes para lo que viene: el esqueleto del proyecto es esa máquina de estados, y en la Entrega 1 necesitarás exactamente estos símbolos Unicode.

Bonus opcional

EJERCICIO 25 • Convierte a match/case [Opcional — Python 3.10+]

Solo si tu versión de Python es 3.10 o superior (compruébalo con `python --version`): coge el ejercicio 2 (Clasifica el número) y reescríbelo usando `match/case` en vez de `if/elif/else`. Si tu versión es anterior, no te preocupes: `if/elif/else` seguirá siendo tu herramienta principal durante el resto del curso.